Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

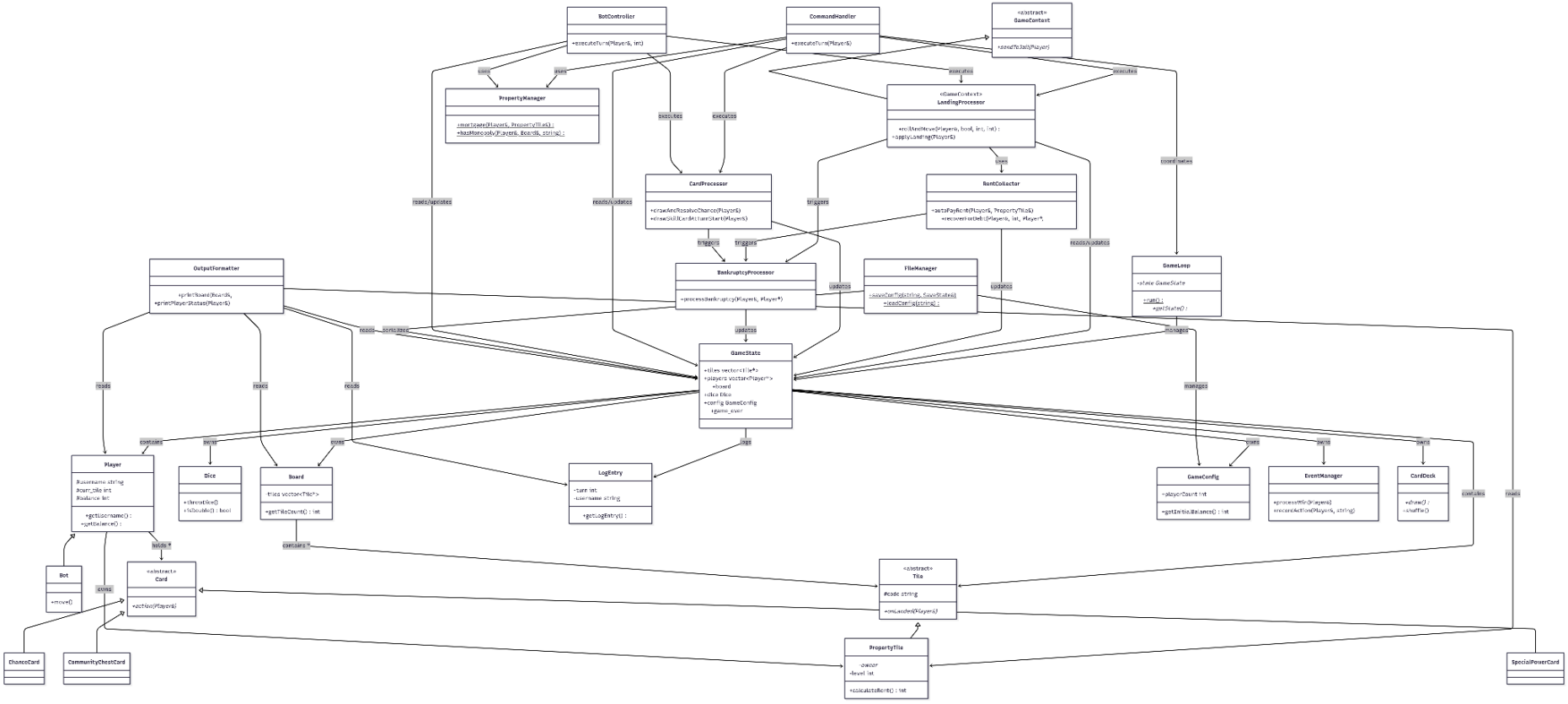
Kode Kelompok : ORI
Nama Kelompok : origami

1. 13524019 / Kevin Wirya Valerian
2. 13524020 / Stevanus Agustaf Wongso
3. 13524023 / Jonathan Kris Wicaksono
4. 13524043 / Emilio Justin
5. 13524101 / Philipp Hamara

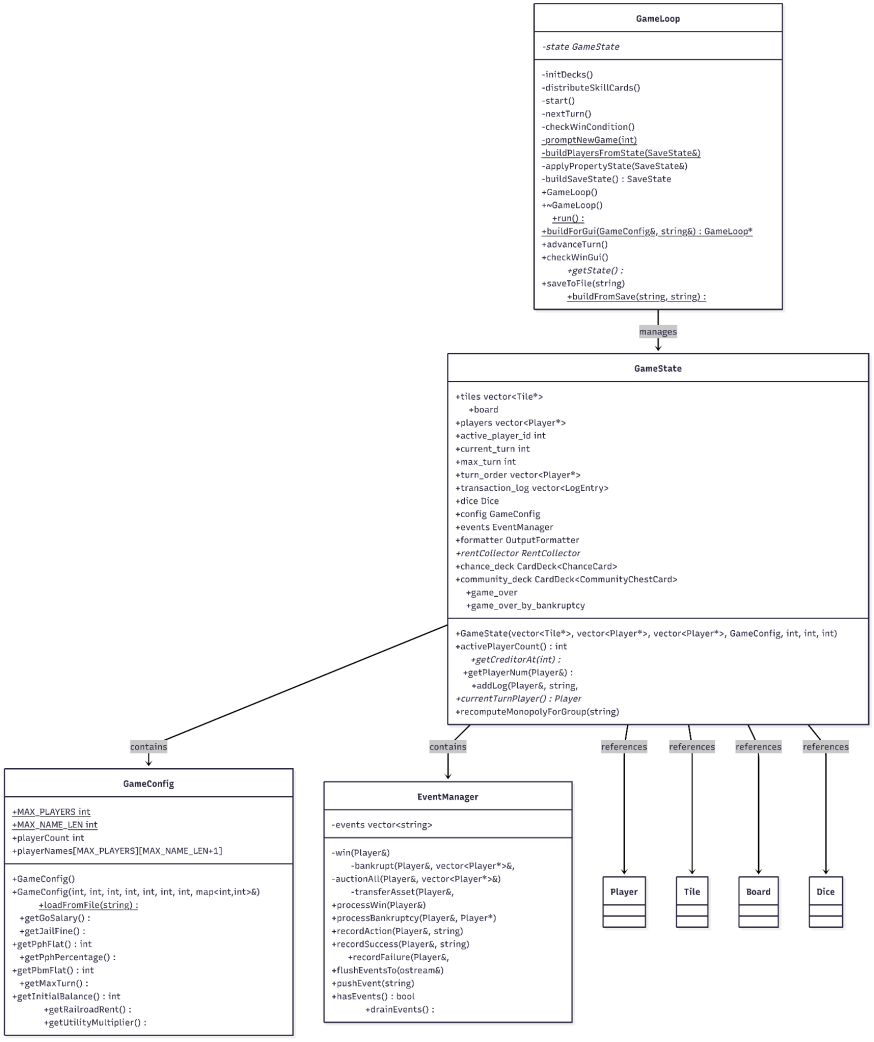
ORI - origami

1. Diagram Kelas

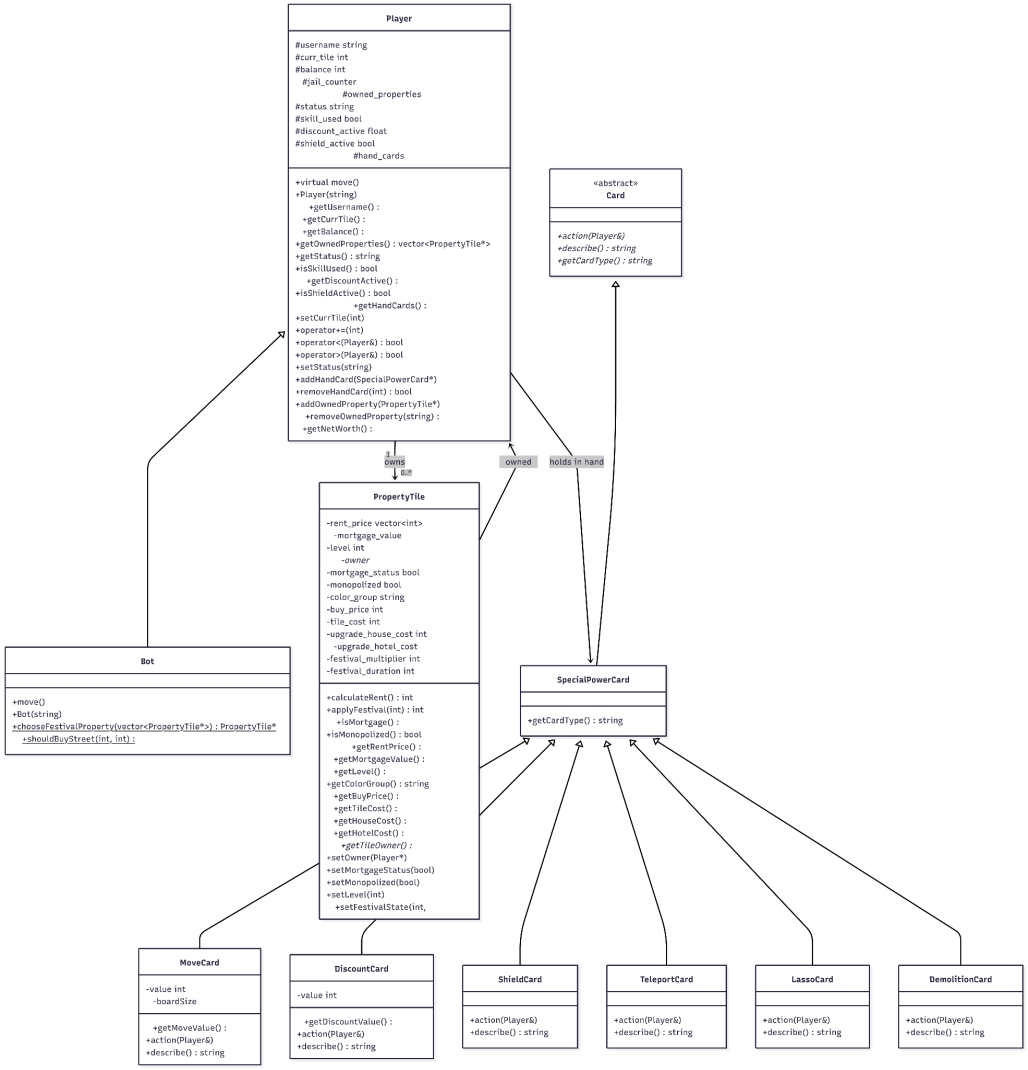
Diagram kelas disusun secara modular untuk meningkatkan keterbacaan dan ketepatan analisis arsitektur sistem: alih-alih menempatkan seluruh kelas dalam satu diagram besar yang padat, setiap diagram memetakan kumpulan kelas yang saling terhubung dalam domain fungsional yang sama, seperti game core, board and movement, card system, financial system, turn execution, serta persistence and output; pendekatan ini membuat relasi antarkelas lebih jelas, memudahkan penelusuran dependensi, mengurangi kompleksitas visual, dan mempercepat proses evaluasi desain, debugging, serta pengembangan lanjutan karena tiap modul dapat ditinjau secara terfokus tanpa kehilangan konteks integrasi sistem secara keseluruhan.



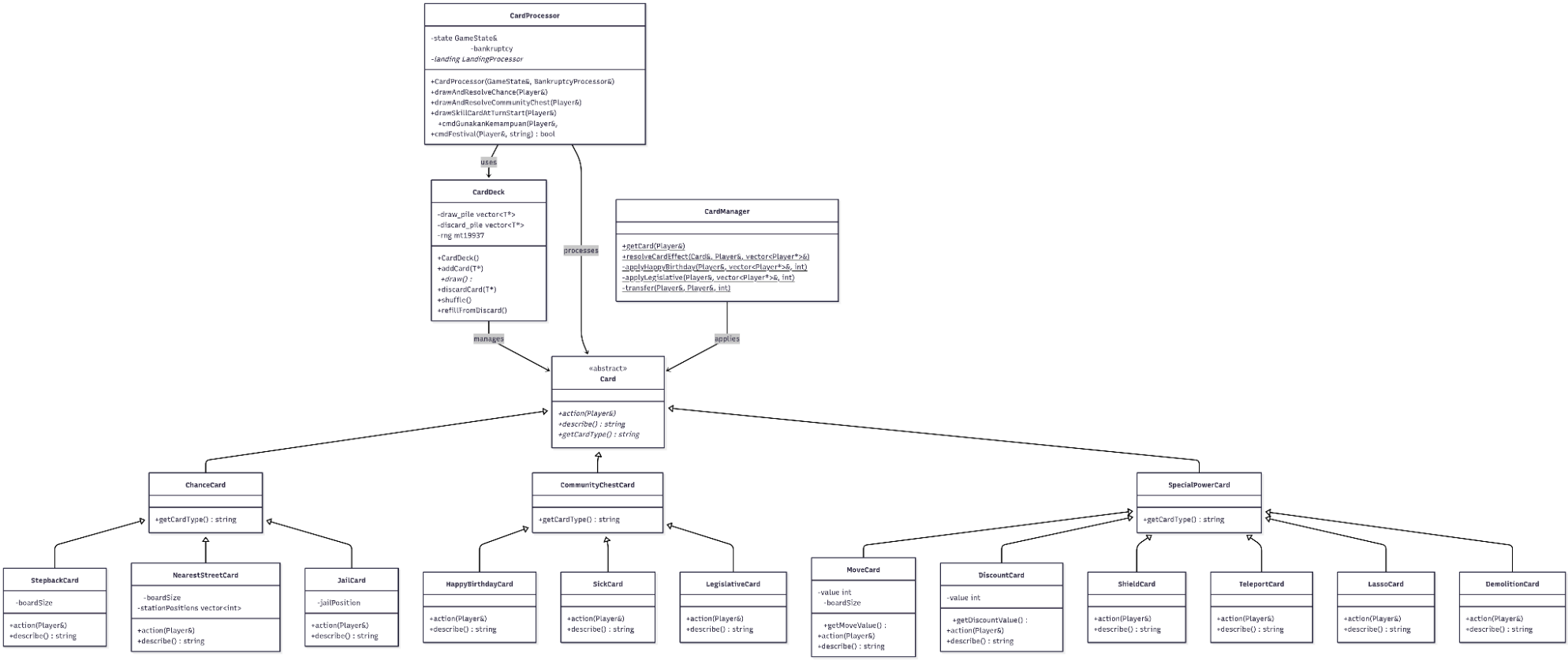
Gambar 1.1 Diagram Kelas System Architecture



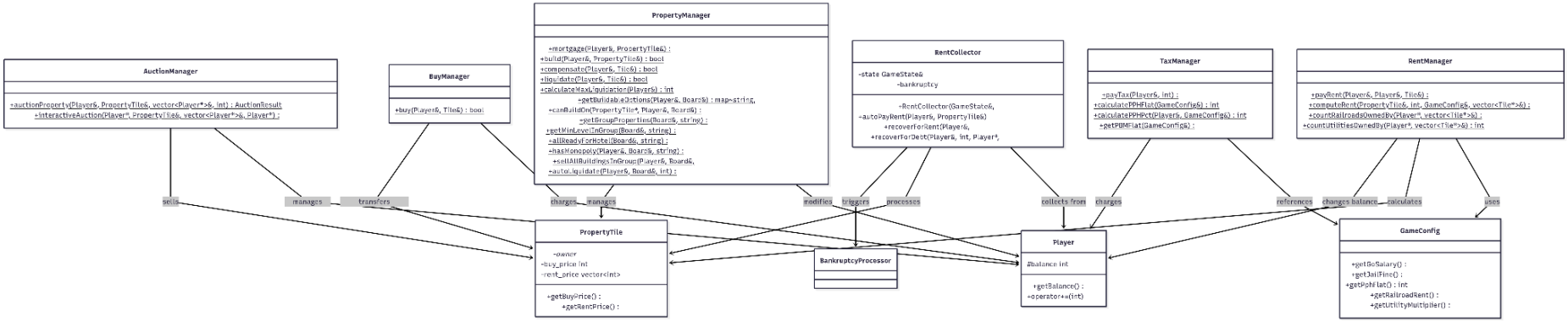
Gambar 1.2 Diagram Kelas Game Core



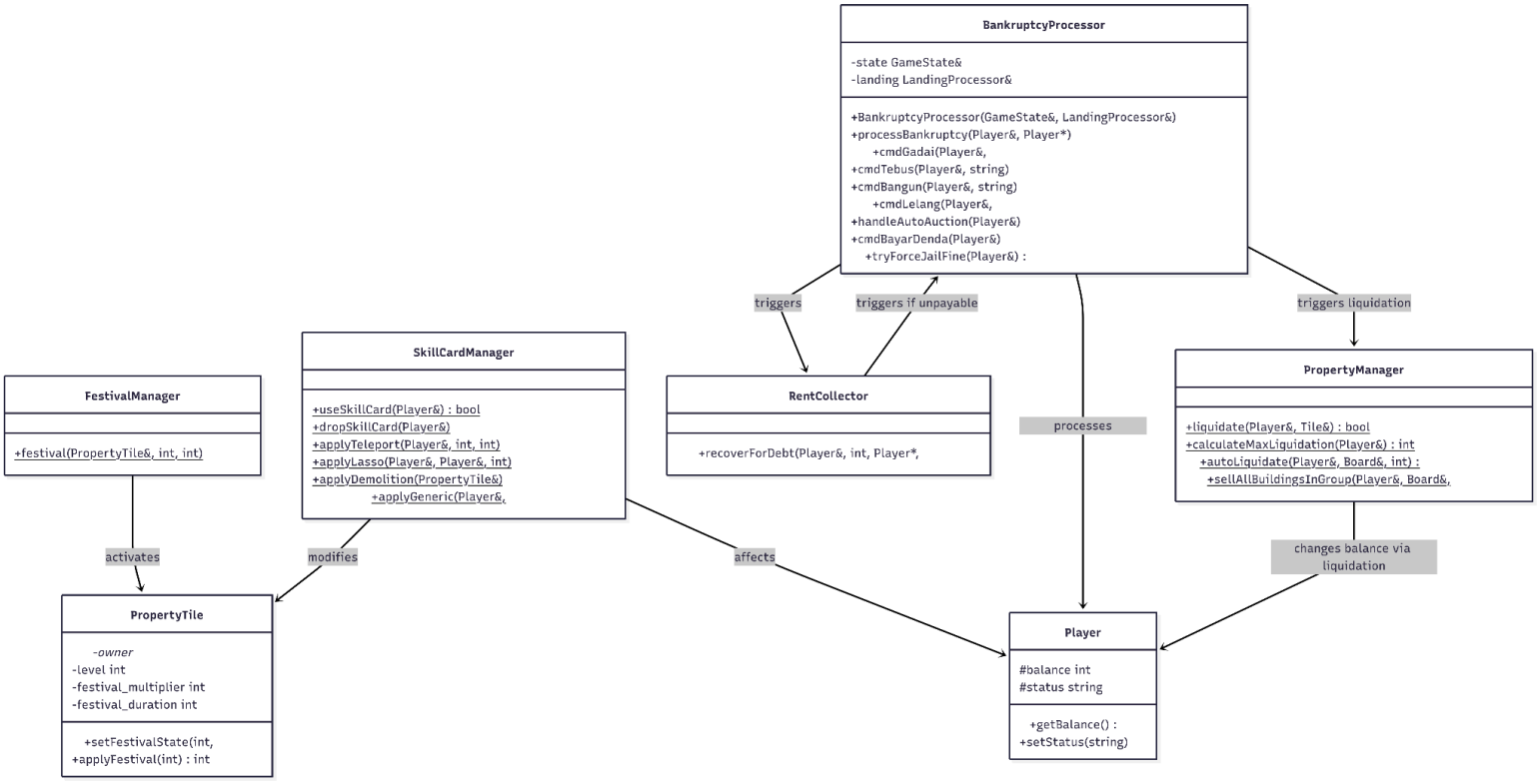
Gambar 1.3 Diagram Kelas Player Assets



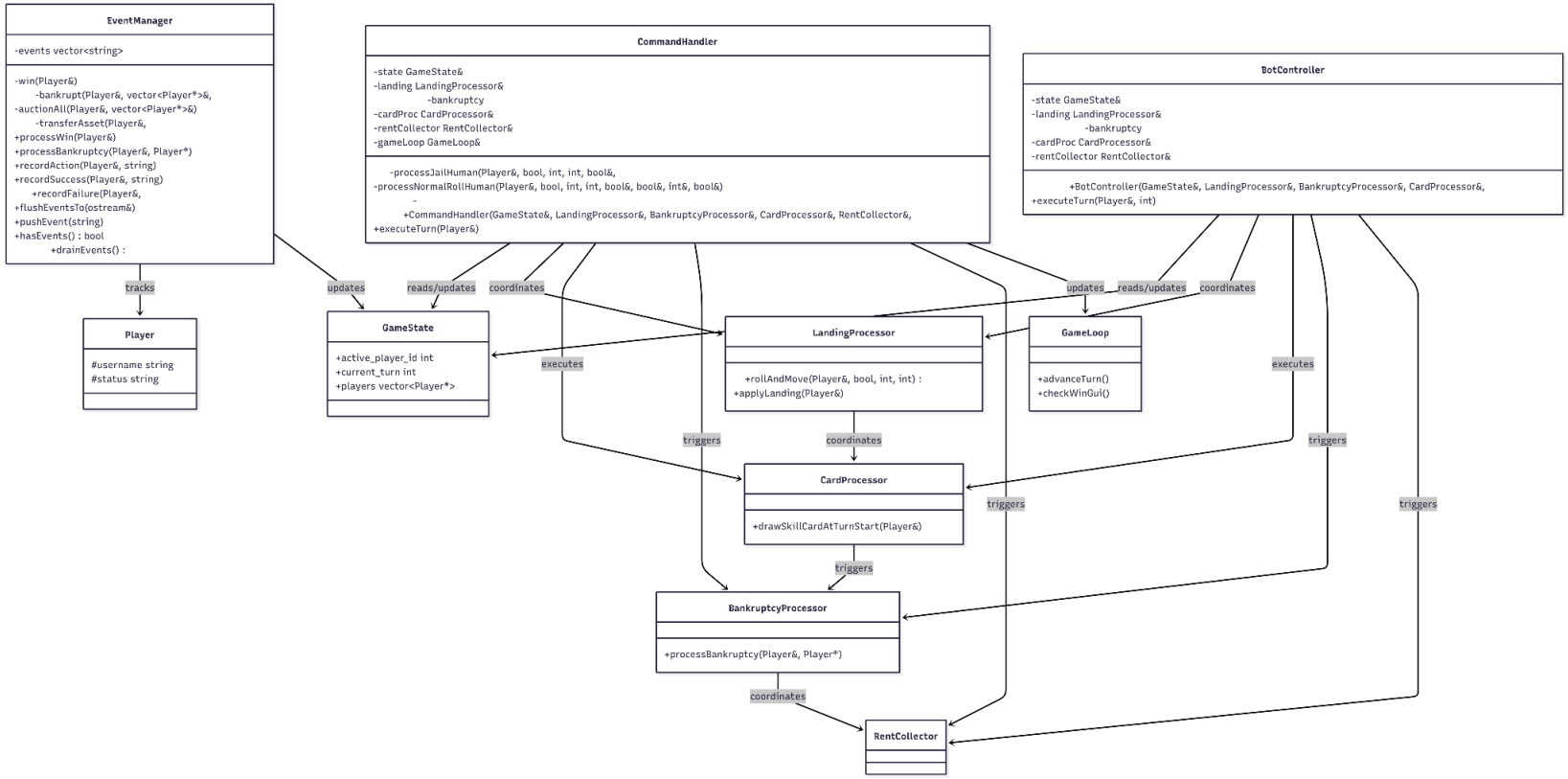
Gambar 1.4 Diagram Kelas Card System



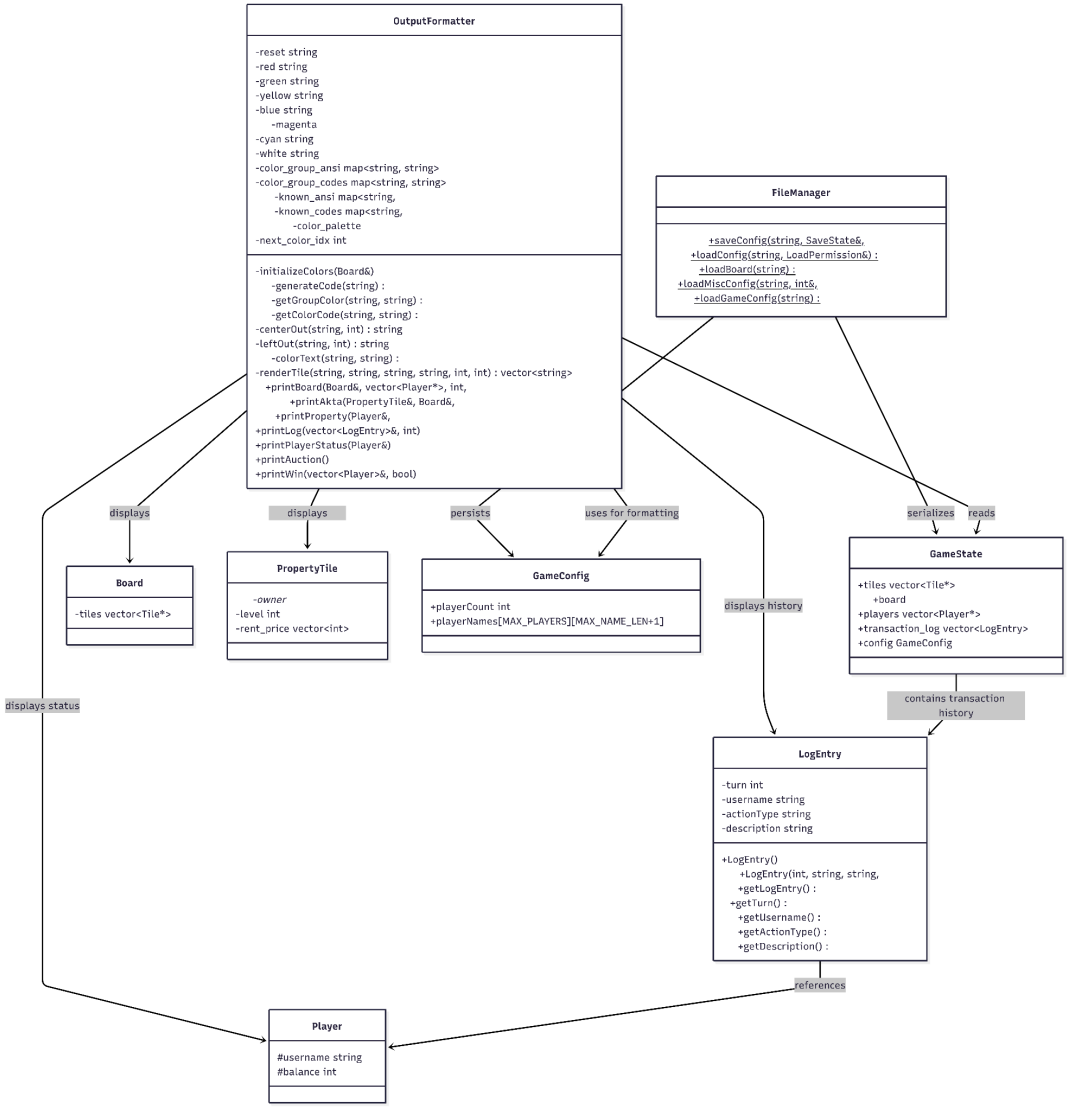
Gambar 1.5 Diagram Kelas System Financial



Gambar 1.6 Diagram Kelas Special Mechanics



Gambar 1.7 Diagram Kelas Turn Execution



Gambar 1.8 Diagram Kelas Persistence & Output

2. Deskripsi Kelas

2.1. Kelas GameLoop

Kelas pusat yang mengorkestrasi keseluruhan alur permainan (sebagai pengganti konsep Origami pada rancangan awal). Mengatur lifecycle permainan melalui method seperti nextTurn(), checkWinCondition(), dan distributeSkillCards(). GameLoop mendelegasikan eksekusi perintah pemain ke CommandHandler dan eksekusi bot ke BotController.

2.2. Kelas GameState

Wadah pusat yang menyimpan seluruh status permainan secara utuh. Atributnya diagregasi dari komponen penting seperti objek Board, vector<Player*>, identitas active_player, current_turn, transaction_log, serta deck tumpukan kartu. GameState diinjeksi ke hampir seluruh kelas manager agar saling berbagi state yang terpusat.

2.3. Kelas GameContext

Kelas interface abstrak yang mendefinisikan kontrak kejadian saat pemain mendarat pada sebuah petak, seperti sendToJail(), drawAndResolveChance(), atau handleTax().

2.4. Kelas LandingProcessor

Mengimplementasikan antarmuka GameContext untuk mengeksekusi logika saat pemain mendarat di suatu petak. Mengatur logika pergerakan pelemparan dadu (rollAndMove) dan pemicuan efek dari petak yang didarati.

2.5. Kelas CommandHandler

Menangani validasi dan eksekusi command dari pemain manusia pada gilirannya (seperti membeli properti, menggunakan kartu, dsb) secara interaktif.

2.6. Kelas BotController

Menangani eksekusi giliran secara otomatis untuk pemain bot. Memiliki hubungan use dengan kelas Bot untuk memanggil algoritma pengambilan keputusan miliknya.

2.7. Kelas BankruptcyProcessor

Menangani proses kebangkrutan pemain secara menyeluruh, serta mengelola komando mekanik finansial yang relevan untuk menghindari kebangkrutan seperti Gadai, Tebus, Bangun, Lelang, dan Bayar Denda.

2.8. Kelas CardProcessor

Bertanggung jawab menghimpun logika penarikan (draw) dan memberikan panggilan instruksi resolusi akhir untuk kartu Chance, Community Chest, dan Skill Card saat dipicu oleh petak atau pemain.

2.9. Kelas RentCollector

Menangani otomatisasi perhitungan, penagihan, dan pembayaran sewa serta hutang (debt). Jika dana pemain tidak mencukupi, kelas ini akan berkoordinasi dengan BankruptcyProcessor untuk memulai proses penagihan aset.

2.10. Kelas Player

Mengatur pergerakan pemain dan menyimpan seluruh state-nya (uang, posisi, properti, kartu skill, status jail). Player memiliki hubungan use dengan Dice dan use dengan CardDeck<T> (saat mengambil/menyimpan kartu). Player menjadi parent dari Bot melalui hubungan is-a.

2.11. Kelas Bot

Mengatur otomatisasi pengambilan keputusan bot (seperti prioritas membeli petak). Memiliki hubungan is-a (inheritance) dengan Player, sehingga seluruh atribut dan method Player dapat dipakai ulang sambil meng-override perilaku decision making.

2.12. Kelas Board

Menyimpan kumpulan petak game melalui atribut vector<Tile*> dan menyediakan utilitas pencarian seperti getTileByCode(), getTileByIndex(), getPropertiesByColorGroup(), dan getColorGroupSize(). Board memiliki hubungan komposisi dengan Tile karena Tile tidak dapat hidup murni tanpa dihubungkan dengan Board.

2.13. Kelas Dice

Menyimpan state dua dadu, melacak double_count, dan menghasilkan nilai lemparan.

2.14. Kelas FileManager

Menangani seluruh I/O file (konfigurasi, save, load). Digunakan untuk memuat state awal, dan membaca parameter konfigurasi permainan dari file eksternal.

2.15. Kelas GameConfig

Menyimpan parameter konfigurasi global (jumlah pemain, batas turn, modal awal, nilai konstan pajak, dll). Memiliki hubungan use dengan FileManager untuk memuat data permainan.

2.16. Kelas LogEntry

Merepresentasikan satu baris transaksi/aksi dalam game. LogEntry diagregasi (has-a) oleh GameState melalui atribut transaction_log: vector<LogEntry> untuk mendukung fitur history.

2.17. Kelas Tile

Kelas abstrak yang menjadi base class seluruh jenis petak. Tile memiliki dua subclass langsung melalui relasi is-a: PropertyTile dan ActionTile. Polymorphism pada Tile memungkinkan Board menyimpan koleksi Tile* tunggal namun tetap dapat memanggil perilaku berbeda (onLanded) saat pemain mendarat.

2.18. Kelas PropertyTile

Merepresentasikan petak properti yang dapat dibeli, dibangun, dan disewakan (termasuk Street, Railroad, dan Utility). Memiliki hubungan is-a dengan Tile.

2.19. Kelas ActionTile

Merepresentasikan petak yang memicu aksi khusus saat didarati. Memiliki hubungan is-a dengan Tile, dan menjadi parent (is-a) dari empat subclass: CardTile, FestivalTile, TaxTile, dan SpecialTile.

2.20. Kelas CardTile

Petak yang memicu pengambilan kartu. Memiliki hubungan is-a dengan ActionTile, dan menjadi parent (is-a) dari ChanceTile dan CommunityChestTile.

2.21. Kelas ChanceTile

Petak yang memicu pengambilan ChanceCard. Memiliki hubungan is-a dengan CardTile.

2.22. Kelas CommunityChestTile

Petak yang memicu pengambilan CommunityChestCard. Memiliki hubungan is-a dengan CardTile.

2.23. Kelas FestivalTile

Petak festival yang menggandakan sewa pada satu warna properti. Memiliki hubungan is-a dengan ActionTile.

2.24. Kelas TaxTile

Petak pajak (mewakili PPh / PBM). Memiliki hubungan is-a dengan ActionTile.

2.25. Kelas SpecialTile

Parent untuk seluruh petak spesial. Memiliki hubungan is-a dengan ActionTile, dan menjadi parent (is-a) bagi empat subclass: StartTile, JailTile, FreeParkingTile, dan GoJailTile.

2.26. Kelas StartTile

Petak Start tempat pemain mendapat gaji setiap melewatinya/mendarat. Hubungan is-a dengan SpecialTile.

2.27. Kelas JailTile

Petak penjara tempat pemain ditahan. Hubungan is-a dengan SpecialTile.

2.28. Kelas FreeParkingTile

Petak parkir gratis tanpa efek aksi yang signifikan. Hubungan is-a dengan SpecialTile.

2.29. Kelas GoJailTile

Petak yang langsung memindahkan paksa pemain ke JailTile tanpa memicu aksi via Start. Hubungan is-a dengan SpecialTile.

2.30. Kelas CardDeck

Kelas template yang mengelola tumpukan kartu generik (draw, shuffle, discard, refill). Penggunaan template parameter generik <T> memungkinkan satu kelas dipakai untuk seluruh deck jenis kartu yang berbeda tanpa duplikasi kode.

2.31. Kelas Card

Base class abstrak seluruh kartu dalam game. Card memiliki tiga subclass langsung melalui hubungan is-a: ChanceCard, CommunityChestCard, dan SpecialPowerCard. Polymorphism memungkinkan setiap kartu mengimplementasikan method action()-nya secara mandiri.

2.32. Kelas ChanceCard

Subclass dari Card. Menjadi parent (is-a) dari NearestStreetCard, StepbackCard, dan JailCard.

2.33. Kelas StepbackCard

Kartu yang memundurkan posisi pemain sejumlah langkah ke belakang. Hubungan is-a dengan ChanceCard.

2.34. Kelas NearestStreetCard

Kartu yang memindahkan pemain secara paksa ke properti tipe stasiun/terdekat. Hubungan is-a dengan ChanceCard.

2.35. Kelas JailCard

Kartu hukuman yang memenjarakan pemain seketika (Get-Into-Jail). Hubungan is-a dengan ChanceCard.

2.36. Kelas CommunityChestCard

Subclass dari Card. Menjadi parent (is-a) dari HappyBirthdayCard, SickCard, dan LegislativeCard.

2.37. Kelas HappyBirthdayCard

Kartu yang membuat semua pemain lain memberi uang ke pemegang kartu. Hubungan is-a dengan CommunityChestCard.

2.38. Kelas SickCard

Kartu yang mengharuskan pemain membayar denda biaya rumah sakit ke bank. Hubungan is-a dengan CommunityChestCard.

2.39. Kelas LegislativeCard

Kartu yang memberikan efek sumbangan kolektif otomatis dari seluruh pemain. Hubungan is-a dengan CommunityChestCard.

2.40. Kelas SpecialPowerCard

Subclass dari Card. Menjadi parent (is-a) bagi seluruh kartu skill: MoveCard, TeleportCard, ShieldCard, LassoCard, DiscountCard, dan DemolitionCard.

2.41. Kelas MoveCard

Kartu skill yang memindahkan pemain sejumlah langkah tertentu sesuai preferensi kartu. Hubungan is-a dengan SpecialPowerCard.

2.42. Kelas DiscountCard

Kartu skill yang memberi status diskon sewaktu membayar sewa atau membeli aset. Hubungan is-a dengan SpecialPowerCard.

2.43. Kelas ShieldCard

Kartu skill yang memberikan penangkal dan melindungi pemain dari tagihan efek negatif satu kali. Hubungan is-a dengan SpecialPowerCard.

2.44. Kelas TeleportCard

Kartu skill yang dapat memindahkan pemain ke petak manapun sesuai keinginannya. Hubungan is-a dengan SpecialPowerCard.

2.45. Kelas LassoCard

Kartu skill yang menarik pemain lain secara paksa ke lokasi target milik pengguna. Hubungan is-a dengan SpecialPowerCard.

2.46. Kelas DemolitionCard

Kartu skill yang dapat menghancurkan dan melenyapkan level bangunan milik properti pemain lain. Hubungan is-a dengan SpecialPowerCard.

2.47. Kelas AuctionManager

Mengelola alur lelang (auction) properti, termasuk validasi peserta dari turn order, serta mekanisme panawaran tawaran lelang. Kelas ini memisahkan logika pelelangan kompleks dari aliran komando utama.

2.48. Kelas BuyManager

Menangani logika pembelian properti oleh pemain saat mendarat pada petak. Memastikan validasi ketersediaan kepemilikan dan pendataan status dana konsisten.

2.49. Kelas CardManager

Mengatur pengambilan kartu dan resolusi efek dari kartu secara utuh menuju pemain. Sebagai titik isolasi proses logika aksi kartu.

2.50. Kelas FestivalManager

Menangani peletakan efek festival dari pemain yang menginjak FestivalTile, memberikan multiplier sewa untuk properti pilihan selama rentang giliran yang valid.

2.51. Kelas JailManager

Mengelola seluruh mekanisme terkait penjara lokal: masuk penjara, lemparan dadu double escape, hukuman durasi kurungan, dan denda keluar bebas dari penjara.

2.52. Kelas PropertyManager

Memusatkan layanan dan validasi aksi atas properti untuk level operasional lanjut: validasi hak gadai (mortgage) atau tebus, algoritma auto liquidate (pembayaran kerugian paksa via jual aset), hingga verifikasi prasyarat monopoli/level properti.

2.53. Kelas RentManager

Sebagai utilitas logika dalam menghakimi berapa besaran nominal sewa murni yang harus difinalisasi saat jatuh di properti tertentu (Street, Railroad, atau Utility).

2.54. Kelas SkillCardManager

Menangani trigger penggunaan kartu skill murni dari inisiasi pemain. Mengkaji validasi kepemilikan dalam batasan pergiliran kartu untuk memicu aksi akhir spesifiknya layaknya Teleport atau Lasso.

2.55. Kelas TaxManager

Mengelola tata cara validasi pembayaran pajak berdasarkan dua ragam perpajakan (PPh untuk persentase pendapatan dan PBM sebagai pajak properti) menyatu atas pedoman konfigurasi.

2.56. Kelas EventManager

Menjalankan dan merekam pencatatan riwayat event yang terjadi ke player (bukan aksi yang dilakukan player). Semua pelaporan dari EventManager dapat dikirimkan ke dalam stream formating Output.

2.57. Kelas GameStates

Menyediakan agregasi spesifikasi struktur wadah data dari status internal game (SaveState, DeckState, PlayerState, PropertyState). Menjadi kontrak mapping save-load oleh FileManager terhadap properti dinamis alur berjalan.

2.58. Kelas OutputFormatter

Menyediakan utilitas statis/helper guna merender template tampilan permainan konsol CLI (mulai dari rendering Tile grid, Log transaksi berwarna, detail properti Pemain, UI form Lelang secara terpusat).

2.59. Kelas GeneralException

Base class abstrak untuk seluruh exception sistem khusus game. GeneralException memiliki struktur hierarki hingga tingkat sub-cucu, guna mempermudah pemanggil melakukan catch exception baik secara spesifik maupun root fail-safe fallback. Ada banyak pewarisan dari kelas ini yang dapat langsung dilihat pada source code program.

3. Justifikasi Desain

Desain kelas kami menerapkan prinsip separation of concerns dengan pembagian tiga lapisan utama: data/state, service/logic, dan orchestration. Pada lapisan data/state, kelas seperti Player, Board, Tile beserta turunannya, Card beserta turunannya, Dice, LogEntry, dan GameConfig berfokus pada representasi objek permainan, penyimpanan state, serta aturan dasar perilaku objek. Pada lapisan service/logic, aturan permainan dipisahkan ke manager yang spesifik agar logika transaksi, pajak, lelang, kartu, skill, dan utilitas tidak bercampur dengan penyimpanan state. Pada lapisan orchestration, alur permainan dikendalikan terutama oleh GameLoop, dibantu CommandHandler,

BotController, dan LandingProcessor untuk sequencing turn, validasi alur aksi, serta koordinasi antarkomponen. Pendekatan ini dipilih karena membuat sistem lebih modular, lebih mudah diuji, dan lebih aman dikembangkan secara bertahap.

Perubahan desain dari milestone 1 terjadi karena saat implementasi kami menemukan bahwa pendekatan satu kelas aksi terpusat terlalu besar dan sulit dipelihara. Oleh karena itu, logika yang semula dibayangkan terkumpul dalam satu ActionManager dipecah menjadi manager-manager yang lebih fokus sesuai domain tanggung jawab (misalnya properti, pajak, lelang, kartu, skill, dan festival), sementara EventManager tetap menangani event level permainan seperti menang dan bangkrut. Selain itu, kebutuhan save/load yang lebih kompleks mendorong penggunaan struktur state yang lebih eksplisit agar proses serialisasi dan deserialisasi lebih konsisten. Perubahan ini menurunkan coupling, mempermudah pengujian per modul, dan mengurangi risiko regresi saat penambahan fitur.

Kelebihan diagram kelas yang dipilih adalah:

1. Inheritance bertingkat pada Tile dan Card mendukung polymorphism, sehingga penambahan jenis petak/kartu baru dapat dilakukan dengan menambah subclass tanpa mengubah banyak kode lama (mendukung Open/Closed Principle).
2. Hierarki exception bertingkat memungkinkan penanganan error yang granular pada kasus spesifik, tetapi tetap bisa ditangkap di level kategori atau umum.
3. Pemisahan FileManager dan OutputFormatter menjaga agar I/O dan presentasi tidak mencemari business logic.

Kekurangan diagram kelas yang dipilih adalah:

1. Banyaknya kelas manager meningkatkan jumlah komponen yang harus dipahami dan diintegrasikan.
2. Interaksi antar kelas service menambah kompleksitas dependency jika tidak dijaga dengan baik.
3. Penyebaran logika ke banyak kelas dapat menambah overhead koordinasi ketika melakukan perubahan lintas fitur.

Kendala utama dalam pemilihan desain OOP adalah menentukan batas tanggung jawab antar kelas agar tidak tumpang tindih, terutama antara aksi turn-level dan event game-level. Kami juga menghadapi trade-off antara desain yang sederhana (lebih cepat diimplementasikan) versus desain yang extensible (lebih tahan perubahan jangka panjang). Selain itu, kami sempat mempertimbangkan apakah kartu cukup direpresentasikan dengan satu kelas bertipe field, namun akhirnya memilih hierarki class agar setiap kartu bisa mengimplementasikan aksi secara polymorphic. Keputusan template untuk deck kartu juga melalui diskusi karena walau lebih abstrak, pendekatan ini memberi keuntungan reuse dan konsistensi implementasi. Secara keseluruhan, desain akhir dipilih karena paling seimbang antara keterbacaan, modularitas, kemudahan pengujian, dan kesiapan pengembangan lanjutan.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Konsep inheritance dan polymorphism dapat terlihat pada pengimplementasian class Tile. Tile terbagi menjadi dua tipe, PropertyTile dan ActionTile. ActionTile dapat dibagi menjadi beberapa tipe. CardTile, FestivalTile, TaxTile, dan SpecialTile mewarisi atribut dan perilaku yang dimiliki oleh Tile. Metode polymorphism diimplementasikan pada fungsi onLanded yang berubah sesuai dengan petak yang diinjak oleh pemain. Alasan kami mengimplementasikan inheritance pada kelas ini adalah kemudahan klasifikasi petak dan kesesuaian dengan spesifikasi yang diberikan. Atribut yang dimiliki oleh ActionTile dan PropertyTile juga berbeda. PropertyTile tentu membutuhkan informasi harga rumah dan hotel, tetapi ActionTile tidak. Namun, keduanya memiliki kesamaan, yaitu memiliki atribut code, id, name, dan type.

Cuplikan dari Tile.hpp

```
#ifndef TILE_HPP
#define TILE_HPP

#include <string>
#include <vector>

class Player;
class GameContext;

class Tile
{
protected:
    std::string code;
    std::string id;
    std::string name;
    std::string type;

public:
    Tile(const std::string &code, const std::string &id, const std::string &name, const std::string &type);
    virtual ~Tile() = default;
    std::string getTileCode() const;
    std::string getTileID() const;
    std::string getTileName() const;
```

```
    std::string getTileType() const;
    virtual void onLanded(Player &player, GameContext &ctx);
};

#endif
```

Cuplikan dari ActionTile.hpp

```
#ifndef ACTIONTILE_HPP
#define ACTIONTILE_HPP

#include "include/models/tiles/Tile.hpp"

class ActionTile : public Tile
{
public:
    ActionTile(const std::string &code, const std::string &id, const std::string &name, const std::string &type);
    void onLanded(Player &player, GameContext &ctx) override;
};

#endif
```

Cuplikan dari PropertyTile.hpp

```
#ifndef PROPERTYTILE_HPP
#define PROPERTYTILE_HPP

#include "include/models/tiles/Tile.hpp"
#include <vector>

class Player;

class PropertyTile : public Tile
{
private:
    std::vector<int> rent_price;
```

```
    int mortgage_value;
    int level;
    Player *owner;
    bool mortgage_status;
    bool monopolized;
    std::string color_group;
    int buy_price;
    int tile_cost;
    int upgrade_house_cost;
    int upgrade_hotel_cost;
    int festival_multiplier;
    int festival_duration;

public:
    PropertyTile(
        const std::string &code, const std::string &id, const std::string &name, const std::string &type,
        std::vector<int> rent_price, int mortgage_value, int level,
        bool mortgage_status, std::string color_group, int buy_price, int tile_cost,
        int upgrade_house_cost, int upgrade_hotel_cost, int festival_multiplier, int festival_duration
    );

    void onLanded(Player &player, GameContext &ctx) override;
    int calculateRent() const;
    int applyFestival(int base_rent) const;
    bool isMortgage() const;
    bool isMonopolized() const;
    std::vector<int> getRentPrice() const;
    int getMortgageValue() const;
    int getLevel() const;
    std::string getColorGroup() const;
    int getBuyPrice() const;
    int getTileCost() const;
    int getHouseCost() const;
    int getHotelCost() const;
    int getFestivalMultiplier() const;
    int getFestivalDuration() const;
```

```
    Player *getTileOwner() const;

    void setOwner(Player *new_owner);
    void setMortgageStatus(bool is_mortgaged);
    void setMonopolized(bool is_monopolized);
    void setLevel(int new_level);
    void setFestivalState(int multiplier, int duration);
};

#endif
```

Hal yang sama juga diimplementasikan pada class Card. Card mewariskan atribut dan methodnya pada tiga kelas turunan, yaitu ChanceCard, CommunityChestCard, dan SpecialPowerCard. Masing-masing kelas turunan ini juga memiliki kelas turunannya sendiri. Misalnya, ChanceCard memiliki kelas turunan StepBackCard, NearestStreetCard, dan JailCard. Pada method action() terjadi polymorphism dengan efek yang bersesuaian dengan deskripsi kartu. Misalnya, kartu StepBackCard yang akan memundurkan pemain sebanyak 3 langkah dan JailCard yang membuat pemain langsung masuk ke dalam penjara. Keuntungan penggunaan konsep inheritance dan polymorphism adalah pengembang dapat dengan mudah mengklasifikasikan suatu dek kartu tersusun dari kartu apa saja. Kemudian, jika ada kartu yang setipe, kita bisa memuat kelas turunan lagi dari masing-masing kartu yang tersedia. Hal ini memudahkan pengembang dalam mengubah efek yang dimiliki kartu tanpa harus mengubah kode program secara keseluruhan. Efisiensi ini akan memotong waktu implementasi kode secara signifikan.

Cuplikan dari Card.hpp

```
#ifndef CARD_HPP
#define CARD_HPP

#include <string>
#include <vector>

class Card {
    public:
        Card();
        virtual ~Card() = default;
        virtual void action(Player& player) = 0;
        virtual std::string describe() const { return ""; }
```

```
};

class ChanceCard : public Card {
    public:
        ChanceCard();
};

class CommunityChestCard : public Card {
    public:
        CommunityChestCard();
};

class SpecialPowerCard : public Card {
    public:
        SpecialPowerCard();
};

#endif
```

Cuplikan dari ChanceCard.hpp

```
#ifndef CHANCECARD_HPP
#define CHANCECARD_HPP

#include "include/models/card/Card.hpp"

class StepbackCard : public ChanceCard {
    private:
        int boardSize;
    public:
        StepbackCard(int boardSize);
        void action(Player& player) override;
        std::string describe() const override;
};
```

```
class NearestStreetCard : public ChanceCard {
    private:
        int boardSize;
        std::vector<int> stationPositions;
    public:
        NearestStreetCard(int boardSize, std::vector<int> stationPositions);
        void action(Player& player) override;
        std::string describe() const override;
};

class JailCard : public ChanceCard {
    private:
        int jailPosition;
    public:
        JailCard(int jailPosition);
        void action(Player& player) override;
        std::string describe() const override;
};

#endif
```

Cuplikan dari CommunityChest.hpp

```
#ifndef COMMUNITYCHEST_HPP
#define COMMUNITYCHEST_HPP

#include "include/models/card/Card.hpp"

class HappyBirthdayCard : public CommunityChestCard {
    public:
        HappyBirthdayCard();
        void action(Player& player) override;
        std::string describe() const override;
};
```

```
class SickCard : public CommunityChestCard {
    public:
        SickCard();
        void action(Player& player) override;
        std::string describe() const override;
};

class LegislativeCard : public CommunityChestCard {
    public:
        LegislativeCard();
        void action(Player& player) override;
        std::string describe() const override;
};

#endif
```

Cuplikan dari SpecialPowerCard.hpp

```
#ifndef SPECIALPOWERCARD_HPP
#define SPECIALPOWERCARD_HPP

#include "include/models/card/Card.hpp"

class MoveCard : public SpecialPowerCard {
    private:
        int value;
        int boardSize;
    public:
        MoveCard(int value, int boardSize);
        int getMoveValue() const;
        void action(Player& player) override;
        std::string describe() const override;
};

class DiscountCard : public SpecialPowerCard {
```

```
    private:
        int value;
    public:
        DiscountCard(int value);
        int getDiscountValue() const;
        void action(Player& player) override;
        std::string describe() const override;
};

class ShieldCard : public SpecialPowerCard {
    public:
        ShieldCard();
        void action(Player& player) override;
        std::string describe() const override;
};

class TeleportCard : public SpecialPowerCard {
    public:
        TeleportCard();
        void action(Player& player) override;
        std::string describe() const override;
};

class LassoCard : public SpecialPowerCard {
    public:
        LassoCard();
        void action(Player& player) override;
        std::string describe() const override;
};

class DemolitionCard : public SpecialPowerCard {
    public:
        DemolitionCard();
        void action(Player& player) override;
        std::string describe() const override;
};
```

```
#endif
```

4.2. Method/Operator Overloading

Konsep method/operator overloading dapat terlihat pada desain dan implementasi class Player. Penggunaan operator overloading pada kelas Player untuk membandingkan net worth serta penggunaan operator += untuk menambah saldo selaras dengan makna alami dari operator tersebut dan meningkatkan keterbacaan kode. Operator perbandingan seperti > atau < secara intuitif merepresentasikan proses membandingkan nilai numerik, sehingga menggunakannya untuk membandingkan kekayaan antar pemain terasa logis dan tidak membingungkan. Demikian pula, operator += secara umum digunakan untuk menambahkan nilai ke suatu variabel, sehingga penerapannya pada penambahan saldo pemain menjaga konsistensi dengan perilaku tipe data bawaan. Selain itu, pendekatan ini membantu menyederhanakan penulisan kode tanpa mengorbankan enkapsulasi, karena detail perhitungan tetap tersembunyi di dalam kelas. Selama implementasinya konsisten dan tidak menimbulkan efek samping yang tidak terduga, penggunaan operator overloading dalam konteks ini merupakan praktik yang tepat dan meningkatkan kualitas desain program.

Cuplikan dari Player.hpp

```
#ifndef PLAYER_HPP
#define PLAYER_HPP

class Player
{
protected:
    int calculateNetWorth() const;

public:
    Player& operator+=(int amount);
    bool operator<(const Player& other) const;
    bool operator>(const Player& other) const;
    bool operator<=(const Player& other) const;
    bool operator>=(const Player& other) const;
};

#endif
```

Cuplikan dari Player.cpp

```
#include "include/models/player/Player.hpp"

Player& Player::operator+=(int amount) { balance += amount; return *this; }
bool Player::operator<(const Player& other) const { return calculateNetWorth() < other.calculateNetWorth(); }
bool Player::operator>(const Player& other) const { return calculateNetWorth() > other.calculateNetWorth(); }
bool Player::operator<=(const Player& other) const { return calculateNetWorth() <= other.calculateNetWorth(); }
bool Player::operator>=(const Player& other) const { return calculateNetWorth() >= other.calculateNetWorth(); }
```

4.3. Template & Generic Classes

Konsep Template dan Generic Classes pada desain CardDeck memberikan type safety yang lebih kuat dibandingkan pendekatan inheritance biasa. Dengan menggunakan Vector<T*> untuk draw_pile dan discard_pile, setiap instansiasi CardDeck hanya akan berisi satu jenis kartu yang spesifik, yaitu ChanceCard atau CommunityChestCard. Hal ini memastikan bahwa tidak dapat terjadi pencampuran tipe kartu dalam satu deck, karena pengecekan dilakukan saat kompilasi, bukan saat runtime. Pada implementasinya di kelas Origami, dua deck yang berbeda dapat dibuat dengan tipe yang berbeda tanpa risiko kesalahan penggunaan. Pendekatan ini lebih tepat dibandingkan menggunakan Vector<Card*> sebagai parent class, karena meskipun fleksibel, pendekatan tersebut membuka kemungkinan tercampurnya berbagai jenis kartu dalam satu deck dan memerlukan validasi tambahan saat runtime. Dengan demikian, penggunaan template tidak hanya meningkatkan keamanan tipe data, tetapi juga membuat desain lebih jelas, terstruktur, dan minim potensi kesalahan.

Cuplikan dari CardDeck.hpp

```
#ifndef CARDDECK_HPP
#define CARDDECK_HPP

#include <vector>
#include <algorithm>
#include <random>
#include <chrono>

template <typename T>
class CardDeck {
    private:
        std::vector<T*> draw_pile;
        std::vector<T*> discard_pile;
        std::mt19937 rng;
```

```
public:
    CardDeck() : rng(std::chrono::steady_clock::now().time_since_epoch().count()) {}

    void addCard(T* card) {
        draw_pile.push_back(card);
    }

    T* draw() {
        if (draw_pile.empty()) {
            refillFromDiscard();
        }
        if (draw_pile.empty()) {
            return nullptr;
        }
        T* card = draw_pile.back();
        draw_pile.pop_back();
        return card;
    }

    T* discard() {
        if (draw_pile.empty()) {
            return nullptr;
        }
        T* card = draw_pile.back();
        draw_pile.pop_back();
        discard_pile.push_back(card);
        return card;
    }

    void discardCard(T* card) {
        if (card != nullptr) {
            discard_pile.push_back(card);
        }
    }
```

```
        void shuffle() {
            std::shuffle(draw_pile.begin(), draw_pile.end(), rng);
        }

        void refillFromDiscard() {
            for (T* card : discard_pile) {
                draw_pile.push_back(card);
            }
            discard_pile.clear();
            shuffle();
        }

        void returnAndShuffle(T* card) {
            if (card != nullptr) {
                draw_pile.push_back(card);
                shuffle();
            }
        }

        bool isEmpty() const {
            return draw_pile.empty();
        }

        int drawPileSize() const {
            return static_cast<int>(draw_pile.size());
        }

        int discardPileSize() const {
            return static_cast<int>(discard_pile.size());
        }
};

#endif
```

Cuplikan dari Origami.hpp

```
#ifndef ORIGAMI_HPP
#define ORIGAMI_HPP

class Origami {
private:
    CardDeck<ChanceCard> chance_deck;
    CardDeck<CommunityChestCard> community_deck;
};

#endif
```

4.4. Exception

Implementasi exception telah kami lakukan pada kelas GeneralException. Kelas ini berperan dalam menghandle seluruh kondisi yang tidak valid ketika player menjalankan suatu perintah. Misalnya, ketika seorang player tidak mampu membayar rent lagi, akan dipanggil UnablePayRentException. Hal ini juga berlaku untuk jenis-jensi Exception lain. Keuntungan penggunaan Exception adalah kita dapat menghandle error itu sendiri daripada tiba-tiba permainan error saat dijalankan. Tanpa exception, kode logika akan bercampur dengan error handling yang membuat bingung. Jika sebuah metode di dalam lapisan terdalam mengalami error, penggunaan exception memudahkan pengembang dalam menangkapnya karena pengembang bisa membiarkan exception tersebut terlempar ke metode pemanggilnya dan akan dilakukan roll-back sewaktu-waktu pengembang ingin mengantisipasi adanya kesalahan.

Cuplikan dari GeneralException.hpp

```
#pragma once

#include <exception>
#include <string>

class GeneralException : public std::exception
{
        protected:
                std::string message;
        public:
                GeneralException();
```

```
                GeneralException(const std::string& msg);
                virtual ~GeneralException() = default;
                const char* what() const noexcept override;
};
```

Cuplikan dari UnablePayRentException.hpp

```
#pragma once

#include "GeneralException.hpp"

class UnablePayRentException : public GeneralException
{
        public:
                UnablePayRentException();
                UnablePayRentException(const std::string& msg);
};
```

4.5. C++ Standard Template Library

4.5.1. std::vector

Cuplikan dari Player.hpp

```
class Player
{
protected:
    std::string username;
    int curr_tile;
    int balance;
    int jail_counter;
    std::vector<PropertyTile *> owned_properties;
    std::string status;
    bool skill_used;
    float discount_active;
    bool shield_active;
```

```
    std::vector<SpecialPowerCard *> hand_cards;
    int calculateNetWorth() const;
.
.
.
};
```

Alasan kami menggunakan std::vector karena jumlah properti milik pemain, kartu di tangan, dan daftar pemain bersifat dinamis (bertambah/berkurang selama permainan). Vector memberikan akses O(1) by index dan cache-friendly karena elemennya kontinu di memori.

4.5.2. std::map

Cuplikan dari GameConfig.hpp

```
class GameConfig
{
.
.
.
    std::map<int, int> railroad_rent;
    std::map<int, int> utility_multiplier;
};
```

Alasan kami menggunakan std::map karena berguna untuk lookup berbasis key. Contohnya, railroad_rent memetakan jumlah stasiun dimiliki (1-4) ke harga sewa yang sesuai. Keuntungannya: pencarian O(log n), key otomatis terurut, dan membuat kode mudah dibaca.

4.5.3. std::set

Cuplikan dari AuctionManager.hpp

```
AuctionManager::AuctionResult AuctionManager::interactiveAuction(Player *seller, PropertyTile &tile, const
std::vector<Player *> &turn_order, Player *trigger) {
.
.
    std::set<Player *> permanentPass;
.
.
};
```

Alasan kami menggunakan std::set karena set menjamin tidak ada duplikat dan menyediakan count() untuk pengecekan keanggotaan dalam O(log n). Pada lelang, set melacak pemain yang sudah pass agar tidak ditanyai lagi.

4.5.4. std::deque

`Cuplikan dari GameScreen.hpp dan GameScreen.cpp`

```
class GameScreen : public Screen {
     std::deque<std::string> eventLog;  // newest at front
}

void GameScreen::pushLog(const std::string& msg) {
    if (msg.empty()) return;
    eventLog.push_front(msg);
    while ((int)eventLog.size() > EVENT_LOG_MAX)
        eventLog.pop_back();
}
```

Alasan kami menggunakan std::deque karena deque mendukung penambahan elemen dari depan dan penghapusan elemen dari belakang yang sangat cocok untuk eventLog.

4.5.5. std::tuple

`Cuplikan dari GameLoop.hpp`

```
class GameLoop {
    static std::tuple<std::vector<Player*>, std::vector<Player*>, int>
        promptNewGame(int initBalance);
    static std::tuple<std::vector<Player*>, std::vector<Player*>, int>
        buildPlayersFromState(const GameStates::SaveState &sstate);
}
```

Alasan kami menggunakan std::tuple karena std::tuple memungkinkan fungsi mengembalikan multiple return values secara ringkas.

4.6. Konsep Tambahan: Generic Class

Implementasi abstract base class dapat terlihat pada class Card yang membuat instansiasi dari kelas Card mustahil semenjak ada metode yang tidak didefinisikan untuk `action(Player& player)` dan `getCardType()`. Hal ini jelas menunjukkan penggunaan abstract base class yang mana suatu kelas tidak dapat diinstansiasi dan butuh instansiasi dari kelas turunannya. Keuntungan dari penggunaan abstract class adalah kemudahan implementasi dengan cara cukup memanggil metode yang ada di class Card, tetapi objek bisa menyesuaikan perilakunya

sesuai dengan kelas turunan yang ada. Akan lebih sulit jika kita langsung menggunakan kelas turunan karena metode bisa disederhanakan dalam suatu kelas abstrak.

Cuplikan Card.hpp

```
#ifndef CARD_HPP
#define CARD_HPP

#include <string>
#include <vector>

class Player;

class Card {
    public:
        Card();
        virtual ~Card() = default;
        virtual void action(Player& player) = 0;
        virtual std::string describe() const { return ""; }
        virtual std::string getCardType() const = 0;
};

class ChanceCard : public Card {
    public:
        ChanceCard();
        std::string getCardType() const override { return "CHANCE"; }
};

class CommunityChestCard : public Card {
    public:
        CommunityChestCard();
        std::string getCardType() const override { return "COMMUNITY"; }
};

class SpecialPowerCard : public Card {
    public:
        SpecialPowerCard();
```

```
        std::string getCardType() const override { return "SKILL"; }
};

#endif
```

Salah satu cuplikan kelas turunan dari ChanceCard.hpp yang menggunakan metode kelas abstrak

```
#ifndef CHANCECARD_HPP
#define CHANCECARD_HPP

#include "include/models/card/Card.hpp"

class StepbackCard : public ChanceCard {
    private:
        int boardSize;
    public:
        StepbackCard(int boardSize);
        void action(Player& player) override;
        std::string describe() const override;
        std::string getCardType() const override { return "STEPBACK"; }
};

class NearestStreetCard : public ChanceCard {
    private:
        int boardSize;
        std::vector<int> stationPositions;
    public:
        NearestStreetCard(int boardSize, std::vector<int> stationPositions);
        void action(Player& player) override;
        std::string describe() const override;
        std::string getCardType() const override { return "NEAREST_STREET"; }
};

class JailCard : public ChanceCard {
    private:
```

```
        int jailPosition;
    public:
        JailCard(int jailPosition);
        void action(Player& player) override;
        std::string describe() const override;
        std::string getCardType() const override { return "JAIL_CARD"; }
};

#endif
```

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. GUI

5.1.1.1. Arsitektur Non-Blocking dengan GUI_MODE

Backend game ini awalnya dirancang untuk CLI, di mana setiap aksi pemain diselesaikan secara blocking melalui stdin.Challange utama dalam GUI adalah perbedaan paradigma: Raylib bekerja dalam game loop yang berjalan sekitar 60 fps dan tidak dapat berhenti di tengah frame untuk menunggu decision player. Digunakan DEFINE flag GUI_MODE agar backend tidak perlu ditulis ulang sepenuhnya. Ketika flag ini aktif, fungsi-fungsi backend yang normalnya memblokir stdin, seperti pengecekan kemampuan bayar sewa atau pajak, diganti untuk melempar exception bertipe spesifik, misalnya UnablePayRentException dan UnablePayPBMTaxException. GUI menangkap exception tersebut melalui blok try-catch di dalam game loop dan menentukan respons visual yang sesuai, tanpa mengubah logika inti permainan.

```
#ifdef GUI_MODE
           // Throw so GUI can show debt recovery UI instead of blocking on stdin
           if (taxType == "TAX_PPH") throw UnablePayPPHTaxException("Pajak PPH M" + to_string(amt));
           if (taxType == "TAX_PBM") throw UnablePayPBMTaxException("Pajak PBM M" + to_string(amt));
#else
           if (rentCollector) {
               rentCollector->recoverForDebt(p, amt, nullptr, "pajak");
           }
           if (p.getStatus() != "BANKRUPT" && p.getBalance() >= amt) {
               p += -amt;
               cout << p.getUsername() << " membayar pajak M" << amt << ".\n";
               state.addLog(p, "BAYAR_PAJAK", taxType + " M" + to_string(amt));
           }
#endif
```

5.1.1.2. State Machine Berbasis Boolean Flag

Setiap decision yang dalam CLI diselesaikan secara langsung, di GUI direpresentasikan sebagai boolean flag state. Flag-flag seperti debtMode, auctionMode, pphPending, dan propertyLandingPending disimpan sebagai atribut layar permainan. Game loop membaca flag-flag ini setiap frame untuk menentukan popup mana yang ditampilkan dan input mana yang aktif, sehingga alur permainan tetap responsif tanpa menghentikan eksekusi program.

```
// ── Auction state ──────────────────────────────────────────────────────────
bool                 auctionMode     = false;
PropertyTile*        auctionProp     = nullptr;
std::vector<Player*> auctionOrder;
int                  auctionIdx      = 0;
int                  auctionHighBid  = 0;
Player*              auctionHighBidder = nullptr;
std::vector<bool>    auctionPassed;
char                 auctionInputBuf[12] = {};
int                  auctionInputLen = 0;
Button               btnAuctionBid;
Button               btnAuctionPass;
```

5.1.1.3. Popup Blocking dengan anyOverlay

Seluruh flag state tersebut digabungkan menjadi satu boolean anyOverlay. Selama ada popup aktif, semua input lain diblokir secara otomatis. Pendekatan ini memastikan hanya satu interaksi yang dapat terjadi dalam satu waktu dan mencegah konflik antar popup yang muncul bersamaan. Dengan ketiga pendekatan ini, backend CLI tetap berjalan utuh untuk mode teks, sementara GUI berjalan di atas lapisan yang sama tanpa duplikasi logika permainan.

```
bool anyOverlay = showPopup || showCardPopup || showSelPopup || gameOverOverlay || setDiceMode
                  || auctionMode || cardBlocks || pphPending
                  || buildMode || festivalPending || propertyLandingPending
                  || saveMode || jailChoicePending || dropCardPending;
```

5.1.2. COM

```
====================================
          NIMONSPOLI
====================================

Path direktori konfigurasi: config

1. New Game
2. Load Game
> 1
Jumlah pemain (2-4): 2

--- Pemain 1 ---
Username: kevin
Tipe (H=Human, B=Bot): B

--- Pemain 2 ---
Username: jo
Tipe (H=Human, B=Bot): B

=== Urutan Giliran ===
1. kevin
2. jo

Game dimulai! Max 15 giliran.

=== Giliran Bot kevin (Turn 1/15) ===
kevin mendapatkan kartu kemampuan baru: ShieldCard - Kebal tagihan/sanksi selama 1 turn
kevin melempar dadu: 3 + 1 = 4
Pindah dari petak 0 ke petak 4.
kevin mendarat di [PPH] Pajak_Penghasilan
Petak Pajak Penghasilan (PPH)
  1. Bayar flat M150
  2. Bayar 10% dari total kekayaan
Bot memilih opsi 2.
Total kekayaan: M1000, pajak: M100
kevin membayar pajak M100.

=== Giliran Bot jo (Turn 1/15) ===
jo mendapatkan kartu kemampuan baru: TeleportCard - Pindah ke petak manapun
jo melempar dadu: 4 + 6 = 10
Pindah dari petak 0 ke petak 10.
jo mendarat di [PEN] Penjara

=== Giliran Bot kevin (Turn 2/15) ===
kevin mendapatkan kartu kemampuan baru: DemolitionCard - Menghancurkan bangunan lawan
kevin melempar dadu: 3 + 6 = 9
Pindah dari petak 4 ke petak 13.
kevin mendarat di [SOL] SOLO
Properti SOLO (SOL) tersedia. Bot akan memutuskan.
kevin (bot) membeli SOLO seharga M140.
```

```
=== Giliran Bot kevin (Turn 9/15) ===
kevin mendapatkan kartu kemampuan baru: DemolitionCard - Menghancurkan bangunan lawan
PERINGATAN: kevin memiliki 4 kartu (Maks 3). Wajib buang 1.
kevin (bot) membuang kartu ke-4.
kevin melempar dadu: 1 + 3 = 4
Pindah dari petak 38 ke petak 2.
kevin melewati GO! Menerima gaji M200.
kevin mendarat di [DNU] Dana_Umum
+=============================+
|       KARTU DANA UMUM       |
+=============================+
| Biaya dokter. Bayar M700.   |
+=============================+
kevin tidak mampu membayar tagihan kartu Dana Umum! Bangkrut!
→ Melelang SOLO (SOL)...
Properti: [SOL] SOLO
Peserta: 1 orang. Minimum bid: M0.
Aksi: PASS atau BID <jumlah>
jo (bot) PASS
Lelang selesai! Pemenang: jo seharga M0
→ Melelang MALANG (MAL)...
Properti: [MAL] MALANG
Peserta: 1 orang. Minimum bid: M0.
Aksi: PASS atau BID <jumlah>
jo (bot) PASS
Lelang selesai! Pemenang: jo seharga M0
→ Melelang STASIUN_TUGU (TUG)...
Properti: [TUG] STASIUN_TUGU
Peserta: 1 orang. Minimum bid: M0.
Aksi: PASS atau BID <jumlah>
jo (bot) BID M50
jo (bot) BID M55
jo (bot) BID M60
jo (bot) BID M65
jo (bot) PASS
Lelang selesai! Pemenang: jo seharga M65
kevin bangkrut!
jo menang!
+================================+
|       PERMAINAN SELESAI!       |
|Semua pemain kecuali satu bangkr|
+================================+

Pemain tersisa:
  - jo

+================================+
|          Pemenang: jo          |
+================================+
```

Pada cuplikan gambar tersebut, terlihat bahwa Nimonspoli dapat dijalankan jika kedua player merupakan bot. Permainan dijalankan secara automatis oleh kedua bot dan sampai pada kondisi kemenangan di akhir. Hal ini kelompok kami implementasikan di CLI dan menampilkan secara lengkap seluruh log kejadian yang terlaksana, membuat apa yang dilakukan Bot sama seperti dilakukan Player. Perbedaannya terletak pada kode implementasinya, Bot melakukan suatu aksi secara *hardcoded*, sedangkan manusia melakukan aksi berdasarkan pilihan yang mereka inginkan.

Berikut adalah beberapa heuristik yang digunakan oleh bot secara keseluruhan.

1. Bot akan membeli properti street hanya jika saldo minimal dua kali harga beli

```
// Bot.cpp
bool Bot::shouldBuyStreet(int balance, int price) {
    return balance >= price * 2;
}
```

```
// BotController.hpp
…
if (prop && !prop->getTileOwner() && !prop->isMortgage() && prop->getTileType() == "STREET") {
        if (Bot::shouldBuyStreet(p.getBalance(), prop->getBuyPrice())) {
            int price = prop->getBuyPrice();
            if (p.getDiscountActive() > 0.0f) {
                price = static_cast<int>(price * (1.0f - p.getDiscountActive()));
            }
            p += -price;
            p.addOwnedProperty(prop);
            state.recomputeMonopolyForGroup(prop->getColorGroup());
            if (p.getDiscountActive() > 0.0f) p.setDiscountActive(0.0f);

            cout << p.getUsername() << " (bot) membeli " << prop->getTileName() << " seharga M" << price <<
".\n";
            state.addLog(p, "BELI", prop->getTileName() + " M" + to_string(price));
        } else {
            cout << p.getUsername() << " (bot) menolak beli. Lelang otomatis.\n";
            bankruptcy.handleAutoAuction(p);
        }
    }
…
```

2. Jika bot mendapatkan efek festival, ia akan memilih properti dengan rent paling besar. Hal ini menggunakan implementasi algoritma greedy

```
// PropertyManager.cpp
int PropertyManager::calculateMaxLiquidation(Player &player) {
    int total = player.getBalance();

    for (auto *p : player.getOwnedProperties()) {
        if (!p) continue;

        if (p->isMortgage()) {
```

```
            // Properti mortgaged harus ditebus dulu (bayar buyPrice) sebelum
            // bisa dijual ke Bank (terima buyPrice). Net = 0.
            continue;
        }

        // Non-mortgaged: max per properti = max(jual ke Bank, gadai).
        //   Jual: terima buyPrice + setengah biaya bangunan (street saja).
        //   Gadai: terima mortgageValue.
        int sellValue = p->getBuyPrice();
        if (p->getTileType() == "STREET" && p->getLevel() > 0) {
            int bldCost = (p->getLevel() >= 5) ? p->getHotelCost() : p->getHouseCost() * p->getLevel();
            sellValue += bldCost / 2;
        }
        int mortgageOpt = p->getMortgageValue();
        if (p->getTileType() == "STREET" && p->getLevel() > 0) {
            // Untuk gadai, harus jual bangunan dulu (separuh), lalu terima mortgageValue
            int bldCost = (p->getLevel() >= 5) ? p->getHotelCost() : p->getHouseCost() * p->getLevel();
            mortgageOpt += bldCost / 2;
        }
        total += (sellValue > mortgageOpt) ? sellValue : mortgageOpt;
    }

    return total;
}

// BotControlerr.cpp
...
else {
                int maxLiq = PropertyManager::calculateMaxLiquidation(p);
                if (maxLiq + p.getBalance() >= fine) {
                    PropertyManager::autoLiquidate(p, state.board, fine);
                    if (p.getBalance() >= fine) {
                        p += -fine;
                        p.setStatus("ACTIVE");
                        state.jail_turns.erase(&p);
```

```
                        cout << p.getUsername() << " (bot) bayar denda M" << fine << ".\n";
                        state.addLog(p, "BAYAR_DENDA", "Paksa keluar M" + to_string(fine));

                        bool isDouble = landing.rollAndMove(p, false);
                        landing.applyLanding(p);

                        if (p.getStatus() == "BANKRUPT") {
                            return;
                        }
                        if (isDouble && p.getStatus() != "JAIL" && p.getStatus() != "BANKRUPT") {
                            executeTurn(p, 0);
                        }
                    } else {
                        bankruptcy.processBankruptcy(p);
                    }
                }
…
```

3. Bot akan mengikuti lelang dengan menaikkan nilai bid tertinggi sebelumnya ditambahkan dengan 5 dan dengan kondisi bahwa nilai bid kurang dari 60% saldo yang ia miliki.

```
// AuctionManager.cpp
if (isBot) {
            int proposed = anyBid ? highestBid + 5 : std::max(1, tile.getMortgageValue());
            if (proposed > cur->getBalance() * 3 / 5) {
                std::cout << cur->getUsername() << " (bot) PASS\n";
                consecPasses++;
            } else {
                proposed = std::min(proposed, cur->getBalance());
                if (proposed <= highestBid) {
                    proposed = highestBid + 1;
                }
                if (proposed > cur->getBalance()) {
                    std::cout << cur->getUsername() << " (bot) PASS\n";
                    permanentPass.insert(cur);
```

```
                    consecPasses++;
                } else {
                    std::cout << cur->getUsername() << " (bot) BID M" << proposed << "\n";
                    highestBid = proposed;
                    highestBidder = cur;
                    anyBid = true;
                    consecPasses = 0;
                }
            }
        }
…
```

4. Kartu kemampuan tetap didapatkan oleh bot dan dibuang, tetapi ia tidak akan menggunakannya.

```
// CardProcessor.cpp

f (p.getHandSize() > 3) {
        cout << "PERINGATAN: " << p.getUsername() << " memiliki " << p.getHandSize() << " kartu (Maks 3).
Wajib buang 1.\n";
        bool isBot = (dynamic_cast<Bot*>(&p) != nullptr);

        if (isBot) {
            int dropIdx = p.getHandSize() - 1;
            cout << p.getUsername() << " (bot) membuang kartu ke-" << (dropIdx + 1) << ".\n";
            p.removeHandCard(dropIdx);
            state.addLog(p, "DROP_KEMAMPUAN", to_string(dropIdx + 1));
        }
…
```

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Papan Permainan	13524019 13524101	13524019 13524020 13524023 13524043 13524101
Player	13254019 13524043 13524101	13524019 13524020 13524023 13524043 13524101
Petak	13524023 13524043	13524019 13524020 13524023 13524043 13524101
Dadu	13524019 13524101	13524019 13524020 13524023 13524043 13524101
Giliran	13524043 13524101	13524019 13524020 13524023 13524043

		13524101
Properti	13524023 13524043	13524019 13524020 13524023 13524043 13524101
Sewa	13524020 13524023	13524019 13524020 13524023 13524043 13524101
Gadai	13524020 13524023	13524019 13524020 13524023 13524043 13524101
Upgrade	13524020 13524043	13524019 13524020 13524023 13524043 13524101
Lelang	13524020 13524043	13524019 13524020 13524023 13524043 13524101

Pajak	13524020 13524023	13524019 13524020 13524023 13524043 13524101
Festival	13524023 13524101	13524019 13524020 13524023 13524043 13524101
Kartu Kesempatan dan Dana Umum	13524043 13524020 13524019	13524019 13524020 13524023 13524043 13524101
Kartu Kemampuan	13524043 13524019 13524043	13524019 13524020 13524023 13524043 13524101
Bangkrut/Pengambilalihan Aset	13524020 13524043	13524019 13524020 13524023 13524043 13524101
Kondisi Game Selesai	13524020	13524019

	13524043	13524020 13524023 13524043 13524101
Save/Load	13524019	13524019 13524020 13524023 13524043 13524101
Transaction Logger	13524020 13524023	13524019 13524020 13524023 13524043 13524101
Exception	13524023 13524019	13524019 13524020 13524023 13524043 13524101
GUI	13524019 13524020 13524023 13524043 13524101	13524019 13524020 13524023 13524043 13524101
COM	13524019 13524020	13524019 13524020

	13524023 13524043 13524101	13524023 13524043 13524101

Lampiran

Lampiran A. Form Asistensi

IF2010 Tugas Besar 1 Asistensi

Lampiran B. Repo GitHub

https://github.com/Labpro-22/tugas-besar-1-ori/tree/main